

Lab: Handling Missing and Inconsistent Data using Pandas in Python

Problem Statement:

Real-world datasets often contain missing or inconsistent data, which can impact data analysis and machine learning models. This lab focuses on identifying, handling, and resolving missing and inconsistent data using Pandas.

Objective:

- Identify missing values in a dataset.
- Handle missing values using different strategies (removal, imputation).
- Detect and resolve inconsistent data.

Requirements:

- Python installed (version 3.x recommended).
- Pandas and NumPy libraries installed (`pip install pandas numpy`).
- Jupyter Notebook (optional but recommended for running the lab).
- Sample dataset with missing and inconsistent data.

Part 1: Importing Libraries and Loading Data

```
In [1]: # import libraries
import pandas as pd
import numpy as np
```

```
In [4]: # Load the dataset
df = pd.read_csv("Dataset/sample_data.csv")
print("Dataset Loaded Successfully")
df.head()
```

Dataset Loaded Successfully

```
Out[4]:
```

	ID	Name	Age	Category	Sales
0	1	Alice	25.0	Electronics	200.0
1	2	Bob	NaN	Electronics	450.0
2	3	Charlie	35.0	Furniture	NaN
3	4	David	40.0	elec	300.0
4	5	NaN	29.0	Office Supplies	500.0

Part 2: Identifying Missing Values

```
In [5]: # Display missing values count per column
print("Missing values per column:")
print(df.isnull().sum())
```

Missing values per column:

ID	0
Name	1
Age	2
Category	0
Sales	1

dtype: int64

Part 3: Handling Missing Values

```
In [6]: # Removing rows with missing values
print("Removing rows with missing values:")
df_dropped = df.dropna()
print(df_dropped.head())
```

Removing rows with missing values:

	ID	Name	Age	Category	Sales
0	1	Alice	25.0	Electronics	200.0
3	4	David	40.0	elec	300.0

```
In [7]: # Filling missing values with a specific value
print("Filling missing values with zero:")
df_filled_zero = df.fillna(0)
print(df_filled_zero.head())
```

Filling missing values with zero:

	ID	Name	Age	Category	Sales
0	1	Alice	25.0	Electronics	200.0
1	2	Bob	0.0	Electronics	450.0
2	3	Charlie	35.0	Furniture	0.0
3	4	David	40.0	elec	300.0
4	5	0	29.0	Office Supplies	500.0

```
In [8]: # Filling missing values with column mean
print("Filling missing values with column mean:")
df_filled_mean = df.fillna(df.mean(numeric_only=True))
print(df_filled_mean.head())
```

Filling missing values with column mean:

	ID	Name	Age	Category	Sales
0	1	Alice	25.00	Electronics	200.0
1	2	Bob	32.25	Electronics	450.0
2	3	Charlie	35.00	Furniture	410.0
3	4	David	40.00	elec	300.0
4	5	NaN	29.00	Office Supplies	500.0

Part 4: Detecting and Resolving Inconsistent Data

```
In [9]: # Checking unique values in a categorical column
df['Category'] = df['Category'].str.strip() # Remove leading/trailing spaces
print("Unique values in 'Category' column before fixing:")
print(df['Category'].unique())
```

Unique values in 'Category' column before fixing:

['Electronics' 'Electronics' 'Furniture' 'elec' 'Office Supplies']

```
In [10]: # Standardizing text values
df['Category'] = df['Category'].replace({'Electronics': 'Electronics', 'elec': 'Electronics'})
print("Unique values in 'Category' column after fixing:")
print(df['Category'].unique())
```

Unique values in 'Category' column after fixing:
['Electronics' 'Furniture' 'Office Supplies']

Part 5: Exporting Cleaned Data

```
In [ ]: # df.to_csv('cleaned_data.csv', index=False)
        # print("Cleaned dataset saved to 'cleaned_data.csv'")
```